

Cấu trúc dữ liệu và giải thuật

Lữ Võ Hoàng Phúc

Ngày 7 tháng 2 năm 2026

Mục lục

1	Thuật toán CSO theo tài liệu	2
2	Thuật toán CSO tối ưu (Khác tài liệu)	8
3	Thuật toán PSO	14

§1 Thuật toán CSO theo tài liệu

Đầu vào:

- ✓ M - số chiều của không gian tìm kiếm
- ✓ $[P_{i,j}^{min}, P_{i,j}^{max}]$ - phạm vi biến thiên cho con mèo thứ i và chiều thứ j
- ✓ MR - tỷ lệ hỗn hợp (mixture ratio)
- ✓ SMP - bộ nhớ gộp chế độ tìm kiếm (seeking memory pool)
- ✓ SRD - phạm vi tìm kiếm của chiều được chọn
- ✓ CDC - số lượng chiều cần thay đổi
- ✓ SPC - cân nhắc vị trí bản thân
- ✓ N - tổng số lượng mèo
- ✓ max - số vòng lặp tối đa
- ✓ c_1 - hằng số cập nhật vận tốc của mèo trong chế độ truy vết

Đầu ra: $Gbest$ - vị trí tốt nhất mà một con mèo đạt được

Thuật toán 4: Phiên bản toàn cục của CSO

```

1  Khởi tạo ngẫu nhiên quần thể mèo  $X_i (i = 1, \dots, N)$  vector  $M$  chiều
2  Khởi tạo  $Gbest$  và gán vị trí tốt nhất ban đầu cho  $Gbest$ 
3  while tiêu chí dừng chưa thỏa mãn hoặc  $I < I_{max}$  do
4      Phân loại mèo vào chế độ tìm kiếm hoặc truy vết dựa trên  $MR$ 
5      Tính toán độ thích nghi (fitness) và cập nhật  $Gbest$ 
6      for  $i \leftarrow 1$  đến  $N$  do
7          if  $X_i$  ở chế độ tìm kiếm then
8              /* Xử lý hành vi tìm kiếm (Seeking Mode) */
9              Tạo  $SMP$  bản sao của  $X_i$ 
10             Cập nhật vị trí:  $X_{cn} = X_c \times (1 \pm SRD \times R)$ 
11             Chọn ngẫu nhiên một vị trí từ  $SMP$  để di chuyển
12         else
13             /* Xử lý hành vi truy vết (Tracing Mode) */
14             Cập nhật vận tốc:  $v_{i,d} = v_{i,d} + R \times c_1 \times (X_{best,d} - X_{i,d})$ 
15             Cập nhật vị trí:  $X_{i,d,new} = X_{i,d,old} + v_{i,d}$ 
16         end
17     end
18 end
19 return  $Gbest$ 

```

```

1  import random
2  import sys

```

```

3
4 class Cat:
5     """
6     Lớp đại diện cho một con mèo (cá thể)
7     """
8     def __init__(self, num_dimensions):
9         self.P = [0.0] * num_dimensions # Vị trí
10        self.V = [0.0] * num_dimensions # Vận tốc
11        self.Pbest = [0.0] * num_dimensions # Vị trí tốt nhất cá nhân
12        self.Eval = 0.0 # Fitness hiện tại
13        self.EvalPbest = float('inf') # Fitness tốt nhất cá nhân
14        self.flag = 0 # 0: Seeking, 1: Tracing
15
16 class CatSwarmOptimization:
17     def __init__(self, M, N, SMP, SRD, CDC, c1, Pmin, Pmax, VMAX, VMIN, MR,
18         ↪ MAX_ITER):
19         """
20         Khởi tạo thuật toán với các tham số được truyền vào.
21
22         Args:
23             M (int): Số chiều không gian
24             N (int): Số lượng mèo (kích thước quần thể)
25             SMP (int): Seeking Memory Pool (Số bản sao trong chế độ tìm kiếm)
26             SRD (float): Seeking Range of the selected Dimension (Phạm vi tìm
27                 ↪ kiếm)
28             CDC (int): Counts of Dimension to Change (Số chiều cần thay đổi)
29             c1 (float): Hằng số gia tốc
30             VMAX (float): Vận tốc tối đa
31             VMIN (float): Vận tốc tối thiểu
32             MR (float): Mixture Ratio (Tỷ lệ hỗn hợp giữa Seeking và Tracing)
33             MAX_ITER (int): Số vòng lặp tối đa
34
35         """
36         # --- 1. CẤU HÌNH THAM SỐ TỪ INPUT ---
37         self.M = M
38         self.N = N
39         self.SMP = SMP
40         self.SRD = SRD
41         self.CDC = CDC
42         self.c1 = c1
43         self.VMAX = VMAX
44         self.VMIN = VMIN
45         self.MR = MR
46         self.MAX_ITER = MAX_ITER
47
48         # Các biến nội bộ
49         self.swarm = [] # Quần thể mèo
50         self.Gbest = [0.0] * M # Vị trí tốt nhất toàn cục
51         self.EvalGbest = float('inf')
52         self.TheBest = 0 # Chỉ số của mèo tốt nhất hiện tại
53
54         # Giới hạn không gian tìm kiếm (Pmin, Pmax)
55         # Bạn có thể đưa cái này ra tham số nếu muốn linh hoạt hơn nữa

```

```

53         self.Pmin = [Pmin] * M
54         self.Pmax = [Pmax] * M
55
56         # --- Các hàm tiện ích ---
57
58         def OF(self, x):
59             """Hàm mục tiêu (Sphere Function)"""
60             t = 0.0
61             for val in x:
62                 t += val * val
63             return t
64
65         def random_0_1(self):
66             """Sinh số ngẫu nhiên [0, 1]"""
67             return random.random()
68
69         # --- Các phương thức chính của thuật toán ---
70
71         def initialize(self):
72             """1. Khởi tạo quần thể"""
73             self.swarm = []
74             for i in range(self.N):
75                 cat = Cat(self.M)
76                 cat.flag = 0
77                 for j in range(self.M):
78                     # Random vị trí trong khoảng [Pmin, Pmax]
79                     cat.P[j] = (self.Pmax[j] - self.Pmin[j]) * self.random_0_1() +
79                                     ↪ self.Pmin[j]
80                     cat.Pbest[j] = cat.P[j]
81                     cat.V[j] = 0.0
82
83                 cat.Eval = self.OF(cat.P)
84                 cat.EvalPbest = cat.Eval
85                 self.swarm.append(cat)
86
87                 # Cập nhật TheBest ban đầu
88                 if self.swarm[i].Eval < self.swarm[self.TheBest].Eval:
89                     self.TheBest = i
90
91                 # Cập nhật Gbest ban đầu
92                 self.EvalGbest = self.swarm[self.TheBest].Eval
93                 self.Gbest = self.swarm[self.TheBest].P[:]
94
95         def assign_modes(self):
96             """2. Phân loại chế độ (Seeking/Tracing)"""
97             cats_in_seeking = int((1 - self.MR) * self.N)
98
99             # Tạo danh sách flag tạm thời
100             temp_flags = [0] * cats_in_seeking + [1] * (self.N - cats_in_seeking)
101             random.shuffle(temp_flags)
102
103             # Gán lại vào mèo

```

```

104         for i in range(self.N):
105             self.swarm[i].flag = temp_flags[i]
106
107     def run_seeking_mode(self, i):
108         """3. Xử lý Chế độ Tìm kiếm"""
109         current_cat = self.swarm[i]
110
111         # Chuẩn bị mảng đánh dấu các chiều cần thay đổi
112         dimensions_to_change = [0] * self.M
113         for j in range(self.CDC):
114             dimensions_to_change[j] = 1
115
116         # Tạo và xử lý các bản sao (SMP bản)
117         copies = []
118         for _ in range(self.SMP):
119             # Tạo bản sao vị trí hiện tại
120             copy_p = current_cat.P[:]
121
122             # Xáo trộn mảng dimensions_to_change cho mỗi bản sao
123             random.shuffle(dimensions_to_change)
124
125             # Đột biến bản sao
126             for j in range(self.M):
127                 if dimensions_to_change[j] == 1:
128                     r = self.random_0_1()
129                     if r > 0.5:
130                         copy_p[j] += self.SRD * copy_p[j] * self.random_0_1()
131                     else:
132                         copy_p[j] -= self.SRD * copy_p[j] * self.random_0_1()
133             copies.append(copy_p)
134
135         # Chọn ngẫu nhiên 1 bản sao để cập nhật vị trí mèo
136         selected_idx = random.randint(0, self.SMP - 1)
137         selected_pos = copies[selected_idx]
138
139         for j in range(self.M):
140             current_cat.P[j] = selected_pos[j]
141             # Kiểm tra biên
142             if current_cat.P[j] > self.Pmax[j]:
143                 current_cat.P[j] = self.Pmax[j]
144             elif current_cat.P[j] < self.Pmin[j]:
145                 current_cat.P[j] = self.Pmin[j]
146
147     def run_tracing_mode(self, i):
148         """4. Xử lý Chế độ Truy vết"""
149         current_cat = self.swarm[i]
150         best_cat_pos = self.swarm[self.TheBest].P
151
152         for j in range(self.M):
153             # Cập nhật vận tốc
154             r = self.random_0_1()

```

```

155         current_cat.V[j] += r * self.c1 * (best_cat_pos[j] -
           ↪ current_cat.P[j])
156
157         # Kiểm tra biên vận tốc
158         if current_cat.V[j] > self.VMAX:
159             current_cat.V[j] = self.VMAX
160         elif current_cat.V[j] < self.VMIN:
161             current_cat.V[j] = self.VMIN
162
163         # Cập nhật vị trí
164         current_cat.P[j] += current_cat.V[j]
165
166         # Kiểm tra biên vị trí
167         if current_cat.P[j] > self.Pmax[j]:
168             current_cat.P[j] = self.Pmax[j]
169         elif current_cat.P[j] < self.Pmin[j]:
170             current_cat.P[j] = self.Pmin[j]
171
172     def evaluate(self):
173         """5. Đánh giá và Cập nhật Gbest"""
174         self.TheBest = 0 # Reset để tìm Best cục bộ vòng này
175
176         for i in range(self.N):
177             self.swarm[i].Eval = self.OF(self.swarm[i].P)
178
179             # Cập nhật Pbest cá nhân
180             if self.swarm[i].Eval < self.swarm[i].EvalPbest:
181                 self.swarm[i].EvalPbest = self.swarm[i].Eval
182                 self.swarm[i].Pbest = self.swarm[i].P[:]
183
184             # Tìm TheBest của vòng lặp hiện tại (để dùng cho Tracing vòng sau)
185             if self.swarm[i].Eval < self.swarm[self.TheBest].Eval:
186                 self.TheBest = i
187
188             # Cập nhật Gbest toàn cục
189             if self.swarm[self.TheBest].Eval < self.EvalGbest:
190                 self.EvalGbest = self.swarm[self.TheBest].Eval
191                 self.Gbest = self.swarm[self.TheBest].P[:]
192
193     def solve(self):
194         """Hàm chạy chính"""
195         self.initialize()
196
197         iteration = 0
198         while iteration < self.MAX_ITER:
199             iteration += 1
200             self.assign_modes()
201
202             for i in range(self.N):
203                 if self.swarm[i].flag == 0:
204                     self.run_seeking_mode(i)
205             else:

```

```

206             self.run_tracing_mode(i)
207
208             self.evaluate()
209
210             print(f"EvalGBest = {self.EvalGbest}")
211             return self.EvalGbest
212
213     # --- HÀM MAIN ---
214     if __name__ == "__main__":
215         # Thiết lập seed để tái lập kết quả (tùy chọn)
216         # random.seed(42)
217
218         # Định nghĩa các tham số đầu vào (Thay vì const)
219         M_input = 5 # Số chiều
220         N_input = 6 # Số lượng mèo
221         SMP_input = 5 # Seeking Memory Pool
222         SRD_input = 0.2 # Phạm vi tìm kiếm
223         CDC_input = 3 # Số chiều cần thay đổi
224         c1_input = 0.5 # Hằng số gia tốc
225
226         Pmin_input = -5.12
227         Pmax_input = 5.12
228
229         VMAX_input = 10.0
230         VMIN_input = -10.0
231         MR_input = 0.2 # Tỷ lệ hỗn hợp
232         MAX_ITER_input = 100 # Số vòng lặp tối đa
233
234         # Khởi tạo đối tượng thuật toán với đầy đủ các tham số
235         cso = CatSwarmOptimization(
236             M=M_input,
237             N=N_input,
238             SMP=SMP_input,
239             SRD=SRD_input,
240             CDC=CDC_input,
241             c1=c1_input,
242             Pmin=Pmin_input,
243             Pmax=Pmax_input,
244             VMAX=VMAX_input,
245             VMIN=VMIN_input,
246             MR=MR_input,
247             MAX_ITER=MAX_ITER_input
248         )
249
250         # Chạy thuật toán
251         cso.solve()

```

§2 Thuật toán CSO tối ưu (Khác tài liệu)

Thuật toán 4: Phiên bản toàn cục của CSO (Tối ưu hóa)

```

1  Khởi tạo ngẫu nhiên quần thể mèo  $X_i$  ( $i = 1, \dots, N$ ) vector  $M$  chiều
2  Khởi tạo  $Gbest$  và gán vị trí tốt nhất ban đầu cho  $Gbest$ 
3  while tiêu chí dừng chưa thỏa mãn hoặc  $I < I_{max}$  do
4      Phân loại mèo vào chế độ tìm kiếm hoặc truy vết dựa trên  $MR$ 
5      Tính toán độ thích nghi (fitness) và cập nhật  $Gbest$ 
6      for  $i \leftarrow 1$  đến  $N$  do
7          if  $X_i$  ở chế độ tìm kiếm then                                     */
8              /* Xử lý hành vi tìm kiếm (Seeking Mode)
9              Tạo  $SMP$  bản sao của  $X_i$ 
10             Cập nhật vị trí:  $X_{cn} = X_c \times (1 \pm SRD \times R)$ 
11             Đánh giá các bản sao và chọn vị trí tốt nhất để di chuyển
12         else
13             /* Xử lý hành vi truy vết (Tracing Mode)
14             Cập nhật vận tốc:  $v_{i,d} = v_{i,d} + R \times c_1 \times (Gbest_d - X_{i,d})$ 
15             Cập nhật vị trí:  $X_{i,d,new} = X_{i,d,old} + v_{i,d}$ 
16         end
17     end
18 end
19 return  $Gbest$ 

```

```

1  import random
2  import sys
3
4  class Cat:
5      """
6      Lớp đại diện cho một con mèo (cá thể)
7      """
8      def __init__(self, num_dimensions):
9          self.P = [0.0] * num_dimensions # Vị trí
10         self.V = [0.0] * num_dimensions # Vận tốc
11         self.Pbest = [0.0] * num_dimensions # Vị trí tốt nhất cá nhân (ít dùng
12         ⇨ trong CSO gốc nhưng giữ lại theo code bạn)
13         self.Eval = 0.0 # Fitness hiện tại
14         self.EvalPbest = float('inf') # Fitness tốt nhất cá nhân
15         self.flag = 0 # 0: Seeking, 1: Tracing
16
17     class CatSwarmOptimization:
18         def __init__(self, M, N, SMP, SRD, CDC, c1, Pmin, Pmax, VMAX, VMIN, MR,
19         ⇨ MAX_ITER):
20             """
21             Khởi tạo thuật toán với các tham số.
22             """

```

```

21         self.M = M
22         self.N = N
23         self.SMP = SMP
24         self.SRD = SRD
25         self.CDC = CDC
26         self.c1 = c1
27         self.VMAX = VMAX
28         self.VMIN = VMIN
29         self.MR = MR
30         self.MAX_ITER = MAX_ITER
31
32         # Các biến nội bộ
33         self.swarm = [] # Quần thể mèo
34         self.Gbest = [0.0] * M # Vị trí tốt nhất toàn cục
35         self.EvalGbest = float('inf')
36         self.TheBest = 0 # Chỉ số của mèo tốt nhất hiện tại
37
38         # Xử lý giới hạn không gian (Pmin, Pmax)
39         self.Pmin = [Pmin] * M
40         self.Pmax = [Pmax] * M
41
42         # --- Các hàm tiện ích ---
43
44         def OF(self, x):
45             """Hàm mục tiêu (Sphere Function) - Càng nhỏ càng tốt"""
46             t = 0.0
47             for val in x:
48                 t += val * val
49             return t
50
51         def random_0_1(self):
52             """Sinh số ngẫu nhiên [0, 1]"""
53             return random.random()
54
55         # --- Các phương thức chính của thuật toán ---
56
57         def initialize(self):
58             """1. Khởi tạo quần thể"""
59             self.swarm = []
60             for i in range(self.N):
61                 cat = Cat(self.M)
62                 cat.flag = 0
63                 for j in range(self.M):
64                     # Random vị trí trong khoảng [Pmin, Pmax]
65                     cat.P[j] = (self.Pmax[j] - self.Pmin[j]) * self.random_0_1() +
66                               ↪ self.Pmin[j]
67                     cat.Pbest[j] = cat.P[j]
68                     cat.V[j] = 0.0
69
69                 cat.Eval = self.OF(cat.P)
70                 cat.EvalPbest = cat.Eval
71                 self.swarm.append(cat)

```

```

72
73     # Tìm mèo tốt nhất ban đầu
74     if self.swarm[i].Eval < self.swarm[self.TheBest].Eval:
75         self.TheBest = i
76
77     # Cập nhật Gbest ban đầu
78     self.EvalGbest = self.swarm[self.TheBest].Eval
79     self.Gbest = self.swarm[self.TheBest].P[:]
80
81     def assign_modes(self):
82         """2. Phân loại chế độ(Seeking/Tracing)"""
83         cats_in_seeking = int((1 - self.MR) * self.N)
84
85         # Tạo danh sách flag: một phần là 0 (Seeking), còn lại là 1 (Tracing)
86         temp_flags = [0] * cats_in_seeking + [1] * (self.N - cats_in_seeking)
87         random.shuffle(temp_flags)
88
89         # Gán flag cho từng con mèo
90         for i in range(self.N):
91             self.swarm[i].flag = temp_flags[i]
92
93     def run_seeking_mode(self, i):
94         """3. Xử lý Chế độ Tìm kiếm (TỐI ƯU: Chọn bản sao tốt nhất)"""
95         current_cat = self.swarm[i]
96
97         # Mảng đánh dấu chiều sẽ thay đổi (CDC chiều)
98         dimensions_to_change = [0] * self.M
99         for j in range(self.CDC):
100             dimensions_to_change[j] = 1
101
102         # Lưu danh sách các bản sao và điểm số của chúng
103         copies = []
104         copies_fitness = []
105
106         # Tạo SMP bản sao
107         for _ in range(self.SMP):
108             copy_p = current_cat.P[:]
109
110             # Xáo trộn mảng đánh dấu để chọn ngẫu nhiên các chiều cần sửa
111             random.shuffle(dimensions_to_change)
112
113             # Đột biến vị trí bản sao
114             for j in range(self.M):
115                 if dimensions_to_change[j] == 1:
116                     r = self.random_0_1()
117                     # Công thức:  $X_{new} = X_{old} \pm SRD * X_{old} * rand$ 
118                     if r > 0.5:
119                         copy_p[j] += self.SRD * copy_p[j] * self.random_0_1()
120                     else:
121                         copy_p[j] -= self.SRD * copy_p[j] * self.random_0_1()
122
123             # Kiểm tra biên ngay lập tức

```

```

124         for j in range(self.M):
125             if copy_p[j] > self.Pmax[j]:
126                 copy_p[j] = self.Pmax[j]
127             elif copy_p[j] < self.Pmin[j]:
128                 copy_p[j] = self.Pmin[j]
129
130         copies.append(copy_p)
131         copies_fitness.append(self.OF(copy_p))
132
133         # --- PHẦN TỐI ƯU QUAN TRỌNG ---
134         # Tìm bản sao có fitness nhỏ nhất (tốt nhất)
135         best_idx = 0
136         best_val = copies_fitness[0]
137
138         for k in range(1, len(copies_fitness)):
139             if copies_fitness[k] < best_val:
140                 best_val = copies_fitness[k]
141                 best_idx = k
142
143         # Cập nhật mèo sang vị trí tốt nhất đó
144         current_cat.P = copies[best_idx][:]
145         # (Lưu ý: Eval sẽ được tính lại ở hàm evaluate() phía dưới để đồng bộ)
146
147     def run_tracing_mode(self, i):
148         """4. Xử lý Chế độ Truy vết"""
149         current_cat = self.swarm[i]
150
151         # Dùng Gbest để dẫn hướng (Tối ưu hơn dùng P[TheBest] cục bộ)
152         best_pos = self.Gbest
153
154         for j in range(self.M):
155             # Cập nhật vận tốc:  $v = v + c1 * r * (Gbest - x)$ 
156             r = self.random_0_1()
157             current_cat.V[j] += r * self.c1 * (best_pos[j] - current_cat.P[j])
158
159             # Kiểm tra biên vận tốc
160             if current_cat.V[j] > self.VMAX:
161                 current_cat.V[j] = self.VMAX
162             elif current_cat.V[j] < self.VMIN:
163                 current_cat.V[j] = self.VMIN
164
165             # Cập nhật vị trí
166             current_cat.P[j] += current_cat.V[j]
167
168             # Kiểm tra biên vị trí
169             if current_cat.P[j] > self.Pmax[j]:
170                 current_cat.P[j] = self.Pmax[j]
171             elif current_cat.P[j] < self.Pmin[j]:
172                 current_cat.P[j] = self.Pmin[j]
173
174     def evaluate(self):
175         """5. Đánh giá toàn bộ và Cập nhật Gbest"""

```

```

176         self.TheBest = 0 # Reset
177
178         for i in range(self.N):
179             # Tính toán Fitness mới
180             self.swarm[i].Eval = self.OF(self.swarm[i].P)
181
182             # Cập nhật Pbest cá nhân (nếu cần)
183             if self.swarm[i].Eval < self.swarm[i].EvalPbest:
184                 self.swarm[i].EvalPbest = self.swarm[i].Eval
185                 self.swarm[i].Pbest = self.swarm[i].P[:]
186
187             # Tìm con tốt nhất trong vòng lặp này
188             if self.swarm[i].Eval < self.swarm[self.TheBest].Eval:
189                 self.TheBest = i
190
191             # Cập nhật Gbest toàn cục nếu tìm thấy giải pháp tốt hơn lịch sử
192             if self.swarm[self.TheBest].Eval < self.EvalGbest:
193                 self.EvalGbest = self.swarm[self.TheBest].Eval
194                 self.Gbest = self.swarm[self.TheBest].P[:]
195
196     def solve(self):
197         """Hàm chạy chính"""
198         self.initialize()
199
200         print(f"Start Optimization with {self.MAX_ITER} iterations...")
201         iteration = 0
202         while iteration < self.MAX_ITER:
203             iteration += 1
204
205             # 1. Chia nhóm
206             self.assign_modes()
207
208             # 2. Chạy từng chế độ cho từng con mèo
209             for i in range(self.N):
210                 if self.swarm[i].flag == 0:
211                     self.run_seeking_mode(i)
212                 else:
213                     self.run_tracing_mode(i)
214
215             # 3. Đánh giá và cập nhật Gbest
216             self.evaluate()
217
218             # In kết quả từng vòng (tùy chọn)
219             # print(f"Iter {iteration}: Best Fitness = {self.EvalGbest}")
220
221         print(f"Final EvalGBest = {self.EvalGbest}")
222         return self.EvalGbest
223
224     # --- HÀM MAIN ---
225     if __name__ == "__main__":
226         # Thiết lập tham số (như code mẫu của bạn)
227         M_input = 5

```

```

228     N_input = 6
229     SMP_input = 5
230     SRD_input = 0.2
231     CDC_input = 3
232     c1_input = 2.05 # Thường dùng 2.05, code bạn là 0.5 (tùy chỉnh)
233
234     Pmin_input = -5.12
235     Pmax_input = 5.12
236
237     VMAX_input = 10.0
238     VMIN_input = -10.0
239     MR_input = 0.2
240     MAX_ITER_input = 100
241
242     # Khởi tạo thuật toán
243     cso = CatSwarmOptimization(
244         M=M_input,
245         N=N_input,
246         SMP=SMP_input,
247         SRD=SRD_input,
248         CDC=CDC_input,
249         c1=c1_input,
250         Pmin=Pmin_input,
251         Pmax=Pmax_input,
252         VMAX=VMAX_input,
253         VMIN=VMIN_input,
254         MR=MR_input,
255         MAX_ITER=MAX_ITER_input
256     )
257
258     # Chạy
259     cso.solve()

```

§3 Thuật toán PSO

Thuật toán 4: Phiên bản toàn cục của PSO

```

1  Xác định hàm mục tiêu  $D$  chiều  $OF(.)$ 
2  Xác định phạm vi biến thiên cho mỗi chiều thứ  $j$ :  $[P_{i,j}^{min}, P_{i,j}^{max}]$ 
3  Xác định các tham số PSO như  $N$  – số lượng hạt,  $c_1, c_2$  – hệ số học tập
4  Tạo ngẫu nhiên bầy  $P$  gồm  $N$  hạt (mỗi hạt là vector  $D$  chiều)
5  Tạo vector  $Gbest$   $D$  chiều
6  for mỗi hạt thứ  $i$  ( $P_i$ ) trong bầy  $P$  do
7     Tạo hạt  $Pbest_i$  bằng với hạt thứ  $i$  ( $P_i$ )
8     Tạo vận tốc  $V_i$  cho mỗi hạt  $P_i$ 
9     Đánh giá hạt  $P_i$  sử dụng hàm  $OF(.)$ 
10 end
11 Gán hạt  $P_i$  tốt nhất cho  $Gbest$ 
12 while điều kiện dừng chưa thỏa mãn do
13     for mỗi hạt  $i$  trong bầy  $P$  do
14         for mỗi chiều  $j$  do
15             Cập nhật vận tốc  $V_{i,j}$  sử dụng công thức:
16              $V_{i,j} = V_{i,j} + c_1 \cdot r_1 \cdot (Pbest_{i,j} - P_{i,j}) + c_2 \cdot r_2 \cdot (Gbest_j - P_{i,j})$ 
17             Cập nhật vị trí hạt  $P_{i,j}$  sử dụng công thức:
18              $P_{i,j} = P_{i,j} + V_{i,j}$ 
19             if  $P_{i,j} < P_j^{min}$  then
20                  $P_{i,j} = P_j^{min}$ 
21             end
22             if  $P_{i,j} > P_j^{max}$  then
23                  $P_{i,j} = P_j^{max}$ 
24             end
25         end
26         Đánh giá hạt  $P_i$  sử dụng hàm  $OF(.)$ 
27         if  $OF(P_i)$  tốt hơn  $OF(Pbest_i)$  then
28             Gán hạt  $P_i$  cho hạt  $Pbest_i$ 
29         end
30     end
31     Chọn hạt tốt nhất từ bầy  $P$  và gán cho hạt  $T$ 
32     if  $OF(T)$  tốt hơn  $OF(Gbest)$  then
33         Gán hạt  $T$  cho hạt  $Gbest$ 
34     end
35 end
36 return  $Gbest$  như là một kết quả

```

```

1  import random
2  import sys
3

```

```

4     class ParticleSwarmOptimization:
5         def __init__(self, N, MAX_ITER, D, c1, c2, Pmin_val, Pmax_val):
6             """
7             Khởi tạo thuật toán PSO với các tham số đầu vào.
8             """
9             # --- Initialization of the parameters (từ input) ---
10            self.N = N # Kích thước quần thể
11            self.MAX = MAX_ITER # Số vòng lặp tối đa
12            self.D = D # Số chiều
13            self.c1 = c1
14            self.c2 = c2
15            self.Iter = 0
16            self.TheBest = 0
17            self.EvalGbest = float('inf') # Khởi tạo giá trị lớn vô cùng
18
19            # Initialization of the constraints
20            # Trong C++ dùng mảng, ở đây ta tạo list Python tương ứng
21            self.Pmin = [Pmin_val] * D
22            self.Pmax = [Pmax_val] * D
23
24            # Initialization of all data structures needed by PSO
25            self.P = [[0.0] * D for _ in range(N)] # Vị trí
26            self.V = [[0.0] * D for _ in range(N)] # Vận tốc
27            self.Pbest = [[0.0] * D for _ in range(N)] # Vị trí tốt nhất cá nhân
28            self.Eval = [0.0] * N # Giá trị fitness hiện tại
29            self.EvalPbest = [0.0] * N # Giá trị fitness tốt nhất cá nhân
30            self.Gbest = [0.0] * D # Vị trí tốt nhất toàn cục
31
32            # definition of the objective function OF(.)
33            def OF(self, x):
34                t = 0.0
35                for val in x:
36                    t += val * val
37                return t
38
39            # generate pseudo random values from the range [0, 1)
40            def r(self):
41                # Mô phỏng logic (rand()%1000)/1000 của C++
42                # Trong Python random.random() trả về [0.0, 1.0) chuẩn xác hơn
43                return random.random()
44
45            def solve(self):
46                """Hàm thực thi chính của thuật toán"""
47
48                # randomly create swarm P and Pbest; create velocity vectors V
49                for i in range(self.N):
50                    for j in range(self.D):
51                        #  $P[i][j] = (P_{max}[j] - P_{min}[j]) * r() + P_{min}[j]$ 
52                        self.P[i][j] = (self.Pmax[j] - self.Pmin[j]) * self.r() +
53                            ↪ self.Pmin[j]
54                        self.Pbest[i][j] = self.P[i][j]
55                        self.V[i][j] = 0.0

```

```

55
56     # evaluate all the particles in the swarm P
57     self.Eval[i] = self.OF(self.P[i])
58     self.EvalPbest[i] = self.Eval[i]
59
60     # find the best particle in the swarm P
61     # Lưu ý: Lần chạy đầu tiên so sánh với TheBest=0
62     if self.Eval[i] < self.Eval[self.TheBest]:
63         self.TheBest = i
64
65     # assign the best particle to Gbest
66     for j in range(self.D):
67         self.Gbest[j] = self.P[self.TheBest][j]
68     self.EvalGbest = self.Eval[self.TheBest]
69
70     # main program loop
71     while self.Iter < self.MAX:
72         self.Iter += 1
73         self.TheBest = 0 # Reset chỉ số tốt nhất cục bộ đầu vòng lặp
74
75         for i in range(self.N):
76             for j in range(self.D):
77                 # for each particle compute the velocity vector
78                 #  $V[i][j] = V[i][j] + c1*r()*(Pbest[i][j]-P[i][j]) +$ 
79                 #  $\hookrightarrow c2*r()*(Gbest[j]-P[i][j])$ 
80                 r1 = self.r()
81                 r2 = self.r()
82                 self.V[i][j] = self.V[i][j] + \
83                     self.c1 * r1 * (self.Pbest[i][j] -
84                      $\hookrightarrow$  self.P[i][j]) + \
85                     self.c2 * r2 * (self.Gbest[j] - self.P[i][j])
86
87                 # update each particle using velocity vector
88                 self.P[i][j] = self.P[i][j] + self.V[i][j]
89
90                 # check the constraints for each dimension
91                 if self.P[i][j] < self.Pmin[j]:
92                     self.P[i][j] = self.Pmin[j]
93                 if self.P[i][j] > self.Pmax[j]:
94                     self.P[i][j] = self.Pmax[j]
95
96                 # evaluate each particle from the swarm P
97                 self.Eval[i] = self.OF(self.P[i])
98
99                 # Update Pbest
100                 if self.Eval[i] < self.EvalPbest[i]:
101                     self.EvalPbest[i] = self.Eval[i]
102                 # for each particle update its Pbest vector
103                 for j in range(self.D):
104                     self.Pbest[i][j] = self.P[i][j]
105
106                 # Tìm phân tử tốt nhất trong vòng lặp hiện tại để cập nhật Gbest

```

```

    ↪ sau này
105         if self.Eval[i] < self.Eval[self.TheBest]:
106             self.TheBest = i
107
108         # update Gbest vector if better particle is found
109         if self.Eval[self.TheBest] < self.EvalGbest:
110             for j in range(self.D):
111                 self.Gbest[j] = self.P[self.TheBest][j]
112                 self.EvalGbest = self.Eval[self.TheBest]
113
114         # Kết quả cuối cùng
115         print(f"EvalGbest = {self.EvalGbest}")
116
117
118     # --- HÀM MAIN ---
119     if __name__ == "__main__":
120         # Thiết lập seed ngẫu nhiên giống C++ (srand(time(NULL)))
121         # random.seed() # Mặc định Python dùng thời gian hệ thống
122
123         # Nhập tham số cấu hình (Input parameters)
124         # Thay vì const hardcoded, ta định nghĩa biến ở đây để truyền vào class
125         N_input = 100
126         MAX_input = 100
127         D_input = 20
128         c1_input = 2.0
129         c2_input = 2.0
130         Pmin_input = -5.12
131         Pmax_input = 5.12
132
133         # Khởi tạo đối tượng PSO với các tham số động
134         pso_solver = ParticleSwarmOptimization(
135             N=N_input,
136             MAX_ITER=MAX_input,
137             D=D_input,
138             c1=c1_input,
139             c2=c2_input,
140             Pmin_val=Pmin_input,
141             Pmax_val=Pmax_input
142         )
143
144         # Chạy thuật toán
145         pso_solver.solve()

```